Tableaux

En Java, on ne manipule pas directement des tableaux (comme en C ou en Pascal), mais des références sur des tableaux. Ainsi la déclaration `int[] tab;` (la syntaxe `int tab[];` est aussi acceptée) déclare une référence sur un tableau d'entiers. Pour pouvoir utiliser un tableau, il faut donc le créer (de la même façon qu'il faut instancier une classe pour utiliser un objet, et non pas simplement déclarer une référence sur un objet) en précisant sa taille : `tab = new int[20];`. Ceci crée un tableau de 20 entiers, ces entiers étant accessibles par `tab[0]` … `tab[19]`. Contrairement au langage C, il est possible de connaître la taille d'un tableau, en accédant à l'attribut `length` d'un tableau (`System.out.println(tab.length);` affichera 20). Attention : cet attribut est particulier et n'est accessible qu'en lecture (`tab.length = 25`; est interdit).

L'exemple ci-dessous déclare une référence sur tableau d'entiers, qui repère un tableau de 20 entiers, ce tableau contenant les valeurs 0 à 19.

```
int[] tab = new int[20];
for (int i=0; i<tab.length; i++) {
     tab[i] = i;
}
```

Méthode `toString()`

Dans `Object`, la méthode `public String toString()` est déclarée ; elle retourne une chaîne de caractères représentant l'objet : sa classe et son adresse. Lorsqu'un objet quelconque est passé comme paramètre à `System.out.println()`, c'est le résultat de l'appel à `toString()` sur cet objet qui est affiché. Afin de pouvoir afficher simplement l'état des objets du programme, on redéfinit souvent la méthode `toString()` dans les classes qu'on crée. Beaucoup de classes de l'API Java redéfinissent aussi la méthode `toString()`.

Javadoc

Lorsqu'on développe un paquetage ou un ensemble de classes, l'utilitaire du JDK (Java Development Toolkit) Javadoc permet de générer automatiquement une documentation des classes au format HTML à partir d'une analyse du code source et des commentaires inclus dans les fichiers source.

Pour cela les commentaires doivent impérativement commencer par `/**` et se terminer par `*/`. Rappelons que `//` peut aussi servir pour des commentaires sur une seule ligne et que `/*` et `*/` encadrent un commentaire qui peut s'étendre sur plusieurs lignes. Des instructions commençant par le symbole `@` permettent d'enrichir les informations traitées par Javadoc :

```
/* Le commentaire suivant sert à décrire la classe MaClasse.
   Il doit être placé juste avant la déclaration. */
/**
 * La classe MaClasse permet de…
 * @author Toto
 * @version 1.0.2
 */
public class MaClasse {
     // Commentaire pour l'attribut monAttribut
     /**
      * monAttribut indique le nombre de…
      */
     private int monAttribut;
     …

     // Commentaire pour la méthode maMethode
     /**
      * Renvoie vrai si les paramètres…
      * @param  x pour décrire le paramètre x
      * @param  y pour décrire le paramètre y
      * @param  z pour décrire le paramètre z
      * @return pour décrire ce que retourne la méthode
      */
     public boolean maMethode(int x, String y, Vehicule z) {
          …
     }
}
```

D'autres mots-clés existent :

- ***@deprecated*** indique qu'une méthode est obsolète et depuis quand ;
- ***@throws*** indique les exceptions que peuvent lever la méthode et dans quelles conditions ;
- ***@see*** indique d'autres documentation de classes ou de méthodes à consulter pour plus d'informations ;
- ***{@link …}*** permet de créer un lien HTML dans la Javadoc vers une autre classe ou méthode ;
- ***{@inheritDoc}*** permet d'hériter de la documentation de la super-méthode.

L'appel à Javadoc est fait de la façon suivante :

```
javadoc [-author] [-version] [-d ./doc] *.java
```

ou :

```
javadoc [-author] [-version] [-d ./doc] package1 package2 …
```

`-d ./doc` permet de spécifier le répertoire[1] où sera mise la documentation.

La documentation est constituée d'un ensemble de fichiers HTML et elle est accessible à partir du fichier `index.html`. Remarque : les tags HTML comme `<b>…</b>` sont utilisables pour insister sur certaines parties de textes.

[1] Le répertoire `./doc` doit exister.